

Developer Security Remediation Guide

Indoore MDMS Backend API — Fix Before Production

Project: Indoore MDMS Backend API Automation

Generated: 25 June 2026

Developer Security Remediation Guide

Indoore MDMS Backend API — Fix Before Production

Field	Detail
Application	Indoore MDM Backend API (https://api.bestinfra.app/indore/*)
Audience	Backend, Frontend, DevOps developers
Purpose	Actionable security fixes required before production deployment
Standard	OWASP Top 10 (2021) · OWASP API Security Top 10
Date	25 June 2026

How to use this document

This is a **developer implementation guide**, not a QA test plan. Each section contains:

- **What is wrong** — confirmed or high-risk gap on live API
- **What to build** — concrete backend / gateway / frontend changes
- **Where to change** — middleware, auth layer, repository, reverse proxy
- **Done when** — verifiable acceptance criteria (curl, logs, headers)

Work through **P0 first** — those block production. P1 before first customer. P2 within 30 days of release.

Executive summary

The API already has strong foundations: JWT auth, CSRF on mutating routes, structured errors without stack traces, and RBAC on permissions endpoints. **Before production**, developers must close these gaps:

Priority	Issue	Developer owner	Effort
P0	Login / refresh not rate-limited	Backend + Gateway	1–2 days
P0	Data-scope / IDOR not enforced on all modules	Backend	3–5 days
P0	CSRF cookie missing <code>HttpOnly</code>	Backend (auth)	0.5 day
P1	CSP allows <code>'unsafe-inline'</code>	Gateway / Frontend	1–2 days
P1	Session not invalidated after logout / force-logout	Backend (auth)	1 day
P1	File upload validation incomplete	Backend (users)	1–2 days
P1	<code>Permissions-Policy</code> header missing	Gateway	0.5 day
P2	User enumeration via distinct 404 messages	Backend	1 day
P2	npm / Docker SCA in CI	DevOps	1 day
P2	Security audit events not logged consistently	Backend	2 days

No Critical or High exploitable bugs were confirmed in API review. Risk is **missing hardening and inconsistent access control** — fixable before release.

P0 — Must fix before production

DEV-01 — Rate limit authentication endpoints

Problem `POST /indore/auth/login` accepts repeated failed attempts without returning **429 Too Many Requests**. Credential stuffing and password spraying are feasible. Invite endpoint may rate-limit; login does not.

What to build

Implement rate limiting at **API gateway (preferred)** or **auth middleware**:

Endpoint	Suggested limit	Key
<code>POST /auth/login</code>	5–10 failures / IP / 15 min	IP + optional username
<code>POST /auth/refresh</code>	30 / user / 15 min	User ID or session
<code>POST /auth/invite</code>	10 / admin user / hour	Authenticated user ID
<code>POST /auth/invite/accept</code>	5 / IP / hour	IP

Implementation (Express / Fastify middleware example)

```
// Pseudocode – use redis-rate-limit or @fastify/rate-limit
const loginLimiter = ratelimit({
  windowMs: 15 * 60 * 1000,
```

```

max: 10,
keyGenerator: (req) => req.ip + (req.body?.email ?? req.body?.username ?? ""),
handler: (req, res) => res.status(429).json({
  error: { code: "RATE_LIMIT_EXCEEDED", message: "Too many attempts. Try again later." }
}),
});
app.post("/indore/auth/login", loginLimiter, loginHandler);

```

Gateway alternative (Nginx)

```

limit_req_zone $binary_remote_addr zone=login_limit:10m rate=5r/m;
location = /indore/auth/login {
  limit_req zone=login_limit burst=3 nodelay;
  proxy_pass http://api_upstream;
}

```

Also log rate-limit hits to audit log (IP, endpoint, user if known — never log password).

Done when

- 15+ failed logins from same IP within 1 minute → **429** with structured error
- Legitimate user can still login after window expires
- Response includes `Retry-After` header (recommended)

DEV-02 — Enforce data scope on every object-level API (IDOR / BOLA)

Problem Auth proves **who** the user is. Many endpoints do not consistently verify the user may access **this specific** DTR, consumer, meter, or billing record. Asset Management has scope logic; **Consumers, DTRs, Billing, HES Commands, Master Data, Reports** need the same pattern.

What to build

1. **Central scope service** — resolve user's allowed org/network/DTR IDs from role + assignment tables (same source as UI filters).
2. **Repository guard** — every query that fetches by ID must filter by scope:

```

// BAD – trusts client-supplied ID only
async getConsumerById(id: number) {
  return prisma.consumer.findUnique({ where: { id } });
}

// GOOD – scope enforced in query
async getConsumerById(id: number, scope: UserScope) {
  return prisma.consumer.findFirst({
    where: {
      id,
      orgId: { in: scope.orgIds },
      // or networkLookupId in scope.networkIds
    },
  });
}
// Return 404 or 403 if null – pick one policy (see DEV-09)

```

1. **Middleware pattern** — attach `req.scope` after JWT validation; reject early if route param ID $\notin$ scope.

Endpoints to audit (minimum)

Module	Routes	Scope field
Asset Management	<code>/dtr/{id}</code> , hierarchies	<code>networkLookupId</code> , org tree
Consumers	<code>/consumers/*</code>	<code>consumerId</code> , <code>orgId</code>
DTRs	<code>/dtrs/*</code>	<code>dtrId</code> , feeder scope
Billing	<code>/billing/*</code>	<code>consumerId</code>
HES Commands	<code>/hes/commands/*</code>	<code>meterId</code> / consumer link
Users Admin	<code>/users/{uuid}</code>	same org only for non-super-admin
Master Data	write endpoints	role + org
Audit export	<code>/audit-logs/export</code>	admin-only + org scope

Mass assignment — use DTO allowlists on PATCH/PUT. Never bind raw body fields like `role` , `isSuperAdmin` , `permissions` from client without admin check:

```
// Reject or strip privileged fields for non-admin
const allowedFields = viewerCanEdit ? ["displayName", "phone"] : [];
```

Done when

- Viewer token + Admin's DTR ID → **403** or empty result (document chosen policy)
- Operator cannot PATCH another org's consumer
- HES disconnect on out-of-scope meter → **403**
- Code review checklist: no `findUnique({ where: { id } })` without scope filter

DEV-03 — Set `HttpOnly` on CSRF cookie

Problem Live probe: `Set-Cookie: csrf_token=...` has `Secure; SameSite=None` but **no** `HttpOnly` . If XSS ever exists, JavaScript can read the CSRF cookie and forge mutating requests.

What to build

In auth login / CSRF token issuance:

```
res.cookie("csrf_token", token, {
  httpOnly: true,  // ADD THIS
  secure: true,
  sameSite: "none", // re-evaluate: use "lax" or "strict" if no cross-site UI
  path: "/",
  maxAge: sessionTtlMs,
});
```

Important: Client must send CSRF via `x-csrf-token` header (already supported). Do **not** rely on JS reading the cookie if `HttpOnly` is set — header-only is correct.

Re-evaluate `SameSite=None` — required only if frontend is on a different site. If UI and API share parent domain, prefer `SameSite=Lax` or `Strict`.

Done when

- `curl -I https://api.bestinfra.app/indore/auth/login` → `Set-Cookie` includes `HttpOnly`
- All mutating routes still pass with `x-csrf-token` header + cookie
- Frontend does not use `document.cookie` to read CSRF

P1 — Fix before GA / first production customer

DEV-04 — Harden Content-Security-Policy

Problem Current CSP: `script-src 'self' 'unsafe-inline'` and `style-src 'self' 'unsafe-inline'`. Weakens XSS protection on any HTML-serving routes.

What to build

API-only JSON responses — minimal CSP is acceptable:

```
Content-Security-Policy: default-src 'none'; frame-ancestors 'none'
```

If HTML error pages or Swagger UI exist — use nonces:

```
const nonce = crypto.randomBytes(16).toString("base64");
res.setHeader("Content-Security-Policy",
  `default-src 'self'; script-src 'self' 'nonce-${nonce}'; style-src 'self' 'nonce-${nonce}'`);
```

Remove `'unsafe-inline'` from gateway config (Nginx, Cloudflare, API middleware).

Done when Security header scan shows no `'unsafe-inline'` in `script-src` / `style-src`.

DEV-05 — Invalidate sessions on logout and force-logout

Problem After logout or admin force-logout, old access tokens may still work until JWT expiry.

What to build

1. **Session store** (Redis recommended) — map `jti` or `sid` from JWT to active session.
2. **On logout / force-logout** — delete session / add `jti` to denylist until `exp`.
3. **On every request** — auth middleware checks denylist or session exists.
4. **Refresh token rotation** — issue new refresh token; invalidate previous on use (detect reuse → revoke all sessions for user).

```

async function authMiddleware(req, res, next) {
  const payload = verifyJwt(req.headers.authorization);
  if (await tokenDenylist.has(payload.jti)) {
    return res.status(401).json({ error: { code: "SESSION_REVOKED" } });
  }
  next();
}

```

Done when

- User logs out → same access token returns **401** immediately
- Admin force-logout → target user's token **401** on next request
- Reusing old refresh token → **401** and all sessions revoked (if rotation enabled)

DEV-06 — File upload hardening (profile image / presigned URL)

Problem `USERS-PROFILE-IMAGE` validates presence of fields but not abuse: dangerous extensions, path traversal, oversized files, SSRF URLs.

What to build

On **presigned URL generation** endpoint:

Rule	Implementation				
Allowlist extensions	<code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> , <code>.webp</code> only				
Block double extension	Reject if <code>fileName</code> matches <code>`\.(exe\`</code>	<code>php\</code>	<code>svg\</code>	<code>html\</code>	<code>js)/i`</code>
Sanitize filename	<code>path.basename(fileName)</code> — reject <code>..</code> , <code>/</code> , <code>\</code> , null bytes				
Max size	Enforce <code>fileSize</code> ≤ 5 MB (configurable) server-side				
Content-Type on S3 PUT	Presigned policy condition: <code>Content-Type</code> starts with <code>image/</code>				
SSRF	If accepting URLs, block private IP ranges (10.x, 172.16–31, 127.x, 169.254.x)				

```

const ALLOWED_EXT = new Set(["jpg", "jpeg", "png", "webp"]);
function validateUpload(fileName: string, fileSize: number) {
  const base = path.basename(fileName);
  if (base !== fileName || base.includes("..")) throw validationError("Invalid file name");
  const ext = base.split(".").pop()?.toLowerCase();
  if (!ext || !ALLOWED_EXT.has(ext)) throw validationError("File type not allowed");
  if (fileSize > 5 * 1024 * 1024) throw validationError("File too large");
}

```

Done when `.exe`, `photo.jpg.exe`, `../../../../x.png`, `fileSize: 999999999` all return **400** at URL generation.

DEV-07 — Add security headers at gateway

Problem `Permissions-Policy` header is missing. Other headers exist but should be centralized so they cannot drift per route.

What to build

Apply at **reverse proxy / API global middleware**:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=()
Content-Security-Policy: <see DEV-04>
```

Remove `X-Powered-By: Express` (or equivalent) — disable framework banner.

CORS — strict allowlist of frontend origins only; no `*` with credentials.

Done when All `/indore/*` responses include the header set above; OPTIONS preflight from unknown origin fails.

P2 — Hardening (within 30 days of release)

DEV-08 — Input validation on all user-controlled parameters

Problem Sample endpoints reject SQLi/XSS in path params (`400 VALIDATION_ERROR`). Not verified on all 115 API routes.

What to build

1. **Schema validation at boundary** — Zod / Joi / class-validator on every query, path, body (already used in places — enforce globally).
2. **Numeric IDs** — coerce with `z.coerce.number().int().positive()` ; reject strings with special chars before DB.
3. **Search q** — max length, escape or parameterize; never concatenate into SQL.
4. **Prisma / parameterized queries only** — no raw `$queryRaw` with string interpolation.
5. **Pagination caps** — `limit` max 100 (or product default); reject `limit=999999` .

Done when Code review rule: no route handler reads `req.params` / `req.query` without schema validation.

DEV-09 — Reduce user / resource enumeration

Problem `GET /indore/users/{uuid}` returns **404 USER_NOT_FOUND** for unknown UUIDs. Attackers can distinguish valid vs invalid IDs.

What to build

For cross-tenant or unauthorized lookups, return **403 Forbidden** with generic message:

```
{ "error": { "code": "ACCESS_DENIED", "message": "You do not have access to this resource." } }
```

Use **404** only when user is authorized to know the resource namespace but item is missing. Apply same policy to consumers, DTRs, invites.

Done when Viewer querying random UUID gets same response shape as querying valid but out-of-scope UUID.

DEV-10 — Security audit logging

Problem Functional audit logs exist; security events are incomplete.

What to log (never log passwords or full tokens)

Event	Fields
Failed login	timestamp, IP, username/email hash, reason
Rate limit hit	IP, endpoint, user ID if auth
403 on protected resource	user ID, role, resource type, resource ID
Force logout	admin ID, target user ID
HES critical command	user ID, meter ID, command type, result
Bulk export	user ID, module, row count, filters
Role / permission change	admin ID, target, before/after

Store in append-only audit table; restrict read to admin roles.

Done when Failed login attempt appears in audit logs within 1 minute; logs queryable from existing audit API.

DEV-11 — Dependency and secret scanning (DevOps)

What to build in CI pipeline

```
# Example GitHub Actions steps
- run: npm audit --audit-level=high
- uses: aquasecurity/trivy-action@master # Docker image scan
- uses: gitleaks/gitleaks-action@v2      # secret scan
```

Block merge on **Critical** CVE in production dependencies. Rotate any leaked secrets immediately.

Done when PR pipeline fails on Critical npm audit finding; no secrets in git history (scan clean).

Module-by-module developer checklist

Use before marking a module "production ready":

#	Check	Module owner
1	All routes require valid JWT except public auth/invite preview	☐
2	Mutating routes validate CSRF header + cookie	☐
3	Object IDs filtered by user scope in repository layer	☐
4	Admin-only routes check role in middleware, not only UI	☐
5	Request body uses allowlist DTO — no mass assignment	☐
6	Query params validated with schema; <code>limit</code> capped	☐
7	Errors return structured JSON — no stack, SQL, file paths	☐
8	Response does not include password, refresh token, internal keys	☐
9	PII fields appropriate for role (Viewer sees less than Admin)	☐
10	File uploads validated (if module has uploads)	☐

Confirmed findings — developer action summary

ID	Finding	Severity	Developer action
F-01	CSRF cookie missing HttpOnly	Medium	DEV-03
F-02	CSP unsafe-inline	Medium	DEV-04
F-03	IDOR / scope gaps	Medium	DEV-02
F-04	No login rate limit	Medium	DEV-01
F-05	Upload validation weak	Medium	DEV-06
F-06	Permissions-Policy missing	Low	DEV-07
F-07	User enumeration 404	Low	DEV-09
F-08	PII in responses	Low (by design)	DEV-02 scope + field filtering
F-09	Debug / verbose errors	Low	DEV-08 + <code>NODE_ENV=production</code>
F-10	Header config not centralized	Low	DEV-07

OWASP mapping — what developers own

OWASP	Category	Developer responsibility
A01	Broken Access Control	DEV-02, DEV-09 — scope in every query
A02	Cryptographic Failures	TLS at infra; never log tokens; bcrypt/argon2 passwords
A03	Injection	DEV-08 — validation + parameterized queries
A04	Insecure Design	DEV-01 rate limits; DEV-06 upload limits
A05	Security Misconfiguration	DEV-03, DEV-04, DEV-07 — cookies & headers
A06	Vulnerable Components	DEV-11 — npm/Docker scan
A07	Auth Failures	DEV-01, DEV-03, DEV-05 — rate limit, session revoke
A08	Software Integrity	DEV-06 — upload validation; signed deployments
A09	Logging Failures	DEV-10 — security audit events
A10	SSRF	DEV-06 — block internal URLs in upload/webhook handlers

Quick verification commands (after fixes)

Developers can verify locally or on staging:

Rate limit

```
for i in {1..15}; do curl -s -o /dev/null -w "%{http_code}\n" \
  -X POST https://api.bestinfra.app/indore/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"wrong@test.com","password":"wrong"}'; done
# Expect 429 after threshold
```

CSRF cookie flags

```
curl -sI https://api.bestinfra.app/indore/auth/login | grep -i set-cookie
# Expect HttpOnly on csrf_token
```

Security headers

```
curl -sI https://api.bestinfra.app/indore/auth/me | grep -iE "strict-transport|content-security|x-frame"
```

Session revoke

```
# 1. Login → save token
# 2. POST logout
# 3. GET /auth/me with old token → expect 401
```

Production merge gate (developer sign-off)

Before merging `release/*` to production:

#	Item	Dev lead sign-off
1	DEV-01 rate limiting deployed	☐
2	DEV-02 scope audit complete for all modules	☐
3	DEV-03 HttpOnly CSRF cookie	☐
4	DEV-05 logout invalidates token	☐
5	DEV-06 upload validation	☐
6	DEV-07 headers on all routes	☐
7	DEV-08 pagination cap + input schemas	☐
8	DEV-10 failed login in audit log	☐
9	DEV-11 CI blocks Critical CVE	☐
10	Staging pen-test or security review for open P0/P1	☐

This guide is based on live API assessment of `https://api.bestinfra.app/indore/*` (June 2026). Backend repository changes are required in the Indoore API codebase and gateway — not in the Playwright test framework.